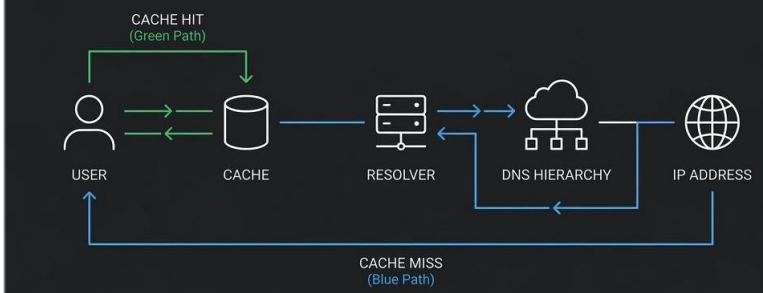
Проект «Кэширование DNS на базе Unbound и Redis» — ключевые темы и подходы

НИУ ВШЭ
Конкурс «Твой проект»
Команда №7

Самойлов М.А. Магелатова В.С. Чванов А.В.

Введение и актуальность кэширования DNS

DNS (Domain Name System) — основа интернета, критична скорость и точность ответов. Кэширование снижает задержки и нагрузку, но TTL ограничивает срок хранения. Требуются методы хранения с произвольным сроком для повышения устойчивости и независимости от внешних серверов.



Архитектура решения

Unbound

DNS-резолвер с возможностью встроенного кеширования и DNSSEC

01

Redis

Внешний кэш DNS-ответов

02

PythonMod

Интеграция Unbound с Redis

03

Сравнительный анализ Unbound и конкурентов

Критерий для проекта	Unbound	PowerDNS	BIND
Гибкое управление TTL и кэшем	**Да**: `cache-min-ttl`, `cache-max-ttl`, `prefetch`, `serve-expired`	Частично	Ограниченно
Поддержка DNSSEC	**Полная**	Полная	Полная
Расширение логики обработки запроса	**PythonMod**	Lua-скрипты	Ограниченно
Интеграция с внешним кэшем Redis	**Удобная**	Возможна	Более сложная
Удобство экспериментальной настройки	**Высокое**	Среднее	Среднее
Подходящесть для нашего проекта	**Наиболее подходящий вариант**	Компромиссный вариант	Менее удобный вариант

Таблица демонстрирует ключевые параметры Unbound по сравнению с основными конкурентами.

Unbound выбран как резолвер с гибкими механизмами кэширования, поддержкой DNSSEC и возможностью расширения через PythonMod для интеграции с Redis.

Сравнительная таблица архитектурных вариантов интеграции с Unbound и Redis

Критерий	Внешний прокси	PythonMod в Unbound	C/C++ плагин
Глубина интеграции	Низкая	**Высокая**	Высокая
Производительность	Средняя	**Высокая**	Максимальная
Сложность реализации	Низкая	**Средняя**	Высокая
Гибкость разработки	Высокая	**Высокая**	Низкая
Доступ к DNS-метаданным	Ограниченный	**Полный**	Полный
Итог для проекта	Компромиссный вариант	**Оптимальный выбор**	Слишком сложен для прототипа

Сравнение выполнено по критериям, важным для проекта: интеграция, производительность и сложность реализации.

PythonMod выбран как оптимальный баланс между глубиной интеграции, производительностью и трудоёмкостью разработки.

Подключение Redis к резолверу

Redis запущен как отдельный сервис в Docker Compose и доступен резолверу по внутренней сети.

01

02

Unbound собран с PythonMod, что позволило встроить логику работы с Redis в обработку DNS-запросов.

Корректность интеграции подтверждена тремя признаками:

redis-cli ping **→ Redis доступен**
unbound -V **→ PythonMod включён**
логи *Unbound* → модуль загружается корректно

Структура DNS-ответа в Redis

Поле	Описание	Пример (yandex.ru)
qname	Доменное имя	yandex.ru.
qtype	Тип записи	A
rcode	Код ответа DNS	NOERROR
answers	Список ответов DNS	[{"name":"yandex.ru.","type":"A","ttl":60,"data":"77.88.44.55"}]
original_ttl	Исходный TTL от сервера	60
stored_at	Время сохранения (unix)	1730000000
admin_expire_at	Административный TTL (unix)	1730086400
source	Источник записи	fill_cache_script
dnssec_status	Статус DNSSEC	unchecked

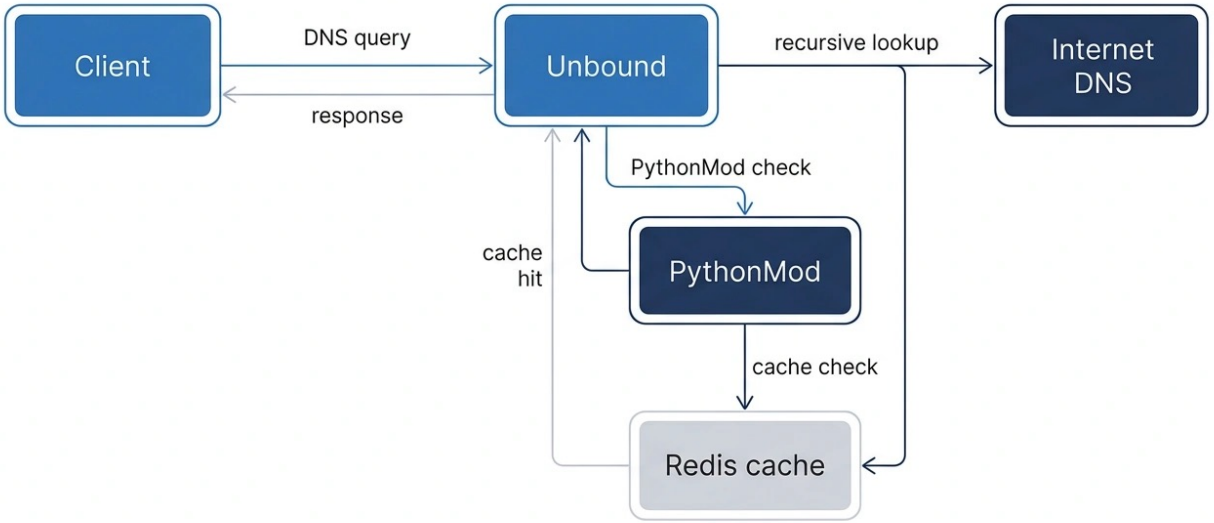
Ключ dns:cache:<fqdn>:<qtype>
Значение Json-объект

Структура позволяет восстановить DNS-ответ, разделить исходный TTL и административное время хранения, а также хранить служебные метаданные.

JSON-структура записей в Redis, эксперимент

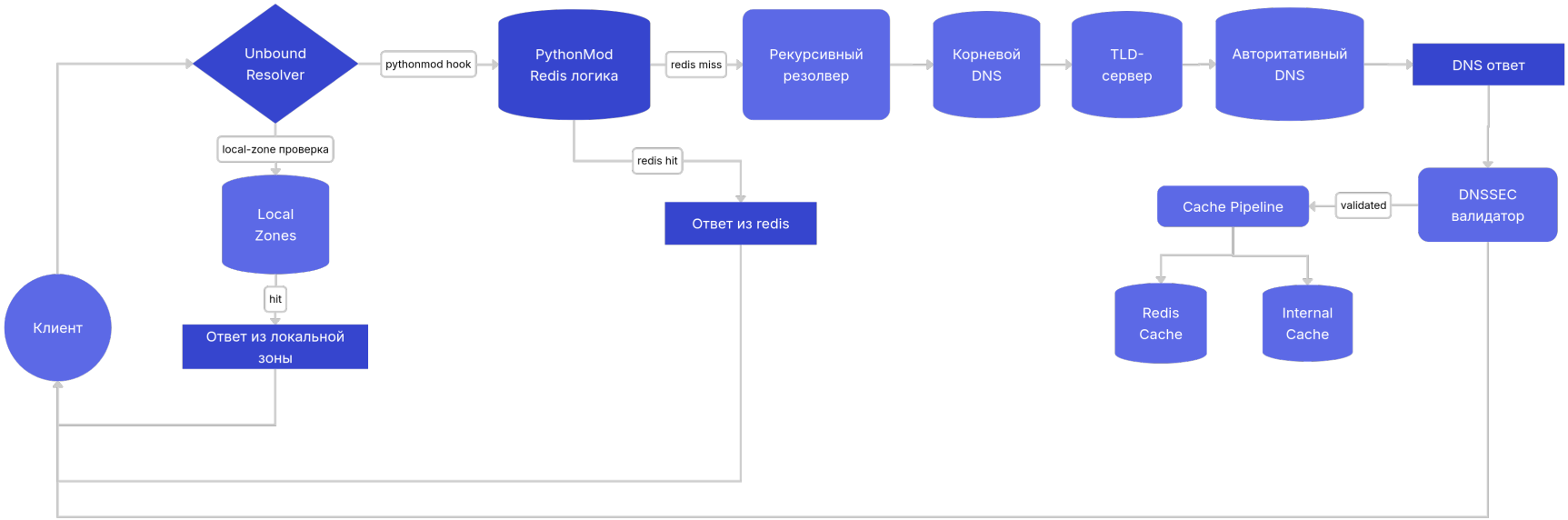
Структура взаимодействия Unbound с Internet, PythonMod и Redis

Взаимосвязи компонентов для реализации кэширования и расширенной обработки DNS-запросов



Обработка DNS-запроса в экспериментальном стенде

Последовательность шагов для обработки запроса в Unbound с интеграцией Redis



Ключевые параметры Unbound и Redis

Параметр	Назначение
`cache-min-ttl`	Увеличивает минимальное время хранения записи в кэше
`cache-max-ttl`	Ограничивает максимальное время хранения записи
`prefetch`	Обновляет часто используемые записи до истечения TTL
`serve-expired`	Позволяет временно отдавать устаревшие записи
`rrset-cache-size`	Задаёт объём кэша DNS-записей
`msg-cache-size`	Задаёт объём кэша готовых DNS-ответов
`module-config`	Подключает PythonMod в цепочку обработки запросов
`REDIS_HOST / PORT / DB`	Настраивают подключение Unbound к Redis

Параметры задают гибкость кэширования, обеспечивают взаимодействие с внешним хранилищем и настраивают обработку TTL.

Конфигурационные файлы Unbound и переменные окружения Docker

(1.1) Методика определения источника DNS-ответа

01

Время ответа

Повторный запрос из кэша выполняется значительно быстрее.

02

Изменение TTL

TTL уменьшается между запросами, если ответ выдан из кэша.

03

Анализ трафика tcpdump

Отсутствие внешних DNS-запросов подтверждает cache hit.

(1.1 A) Базовое кэширование на примере yandex.ru

Повторный запрос через 5 секунд

Ответ из кэша.
2 мс, TTL: 600 → 595

653 мс → 2 мс
TTL: 600 → 595

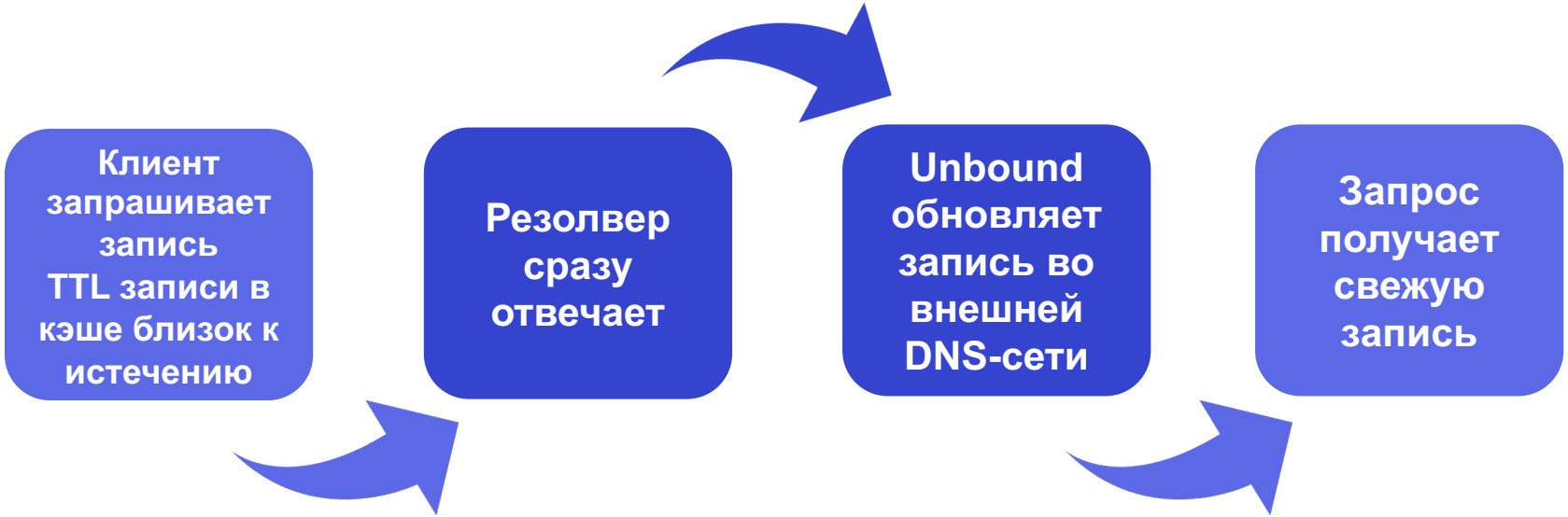
Первый запрос

Рекурсивное разрешение и заполнение кэша.
653 мс

Проверка tcpdump

Внешнего трафика нет

(1.1 Б) Механизм prefetch в Unbound



(1.1 B) Влияние *cache-min-ttl* на кеширование

```
dig @127.0.0.1 -p 2053 samokat.ru A
sleep 400
dig @127.0.0.1 -p 2053 samokat.ru A
```

samokat.ru
TTL = 300s

cache-min-ttl = 0

1-й запрос: **1691 мс**
ожидание: **400 с**

2-й запрос: **1690 мс**
Рекурсия выполняется заново

cache-min-ttl = 600

1-й запрос: **1625 мс**
ожидание: **400 с**

2-й запрос: **1 мс**
Ответ выдан из кэша

(1.1 Г) Вытеснение записей из кеша

```
for i in $(seq 1 1000); do
  dig @127.0.0.1 -p 2053 nx-$i.com A +tries=1 +time=1 > /dev/null 2>&1
  dig @127.0.0.1 -p 2053 nx-$i.com AAAA +tries=1 +time=1 > /dev/null 2>&1
done
```

Параметр	Значение (до)	Значение (после)
cache-min-ttl	60	3600
msg-cache-size	64Мб	128Кб
rrset-cache-size	128Мб	256Кб
cache-min-ttl	86400	86400

(1.2 A) Подключение Redis к резолверу

Подтверждение доступности Redis

Команда *redis-cli ping* возвращает PONG.

Сборка и запуск контейнеров

1. docker compose up -d--build
2. docker compose logs unbound
3. docker exec -it dns-unbound /usr/local/sbin/unbound -V | grep -i python
4. docker exec -it dns-redis redis-cli ping

(1.2 Б) Проверка *redis_hit / redis_miss*

Запись добавляется через fill_cache.py

Redis получает DNS-ответ

Нет записи в Redis

В логах фиксируется **redis_miss**

Повторный запрос

В логах фиксируется **redis_hit**

(1.2 B) Формирование DNS-ответов из Redis

Если запись найдена в Redis, PythonMod извлекает сохранённые данные и передаёт их в Unbound для формирования DNS-ответа.

01

02

При redis_hit ответ формируется напрямую из внешнего кэша, без обращения к корневым, TLD и авторитетным DNS-серверам.

Redis в нашей архитектуре — не только хранилище, но и полноценный источник DNS-ответов.

(1.2 Г) Разделение TTL ответа и TTL хранения

TTL клиента:	**60 с**
TTL в Redis:	**86400 с**

Сохранение в Redis

fill_cache.py сохранил запись с TTL = **86400 с**

Проверка Redis

redis-cli TTL вернул **86366 с**

Проверка клиентского ответа

dig показывает **TTL = 60 с**

Выбор DNSSEC-подписанной зоны и её локальное размещение

Выбрана DNSSEC-подписанная зона

Для эксперимента выбрана зона с поддержкой DNSSEC и размещена локально на резолвере.

01

Резолвер настроен на локальную зону

Запросы обрабатываются локально, без обращения к авторитетным DNS-серверам.

02

Локальный источник ответа подтверждён

Трассировка и проверки показали, что ответы выдаются из локальной базы резолвера.

03

(1.3 Б) Ответы из локальной зоны

Конфигурация

local-zone: "cloudflare.com." static
local-data: "cloudflare.com. 300 IN A 192.0.2.1"

Публичный DNS (8.8.8.8)

Возвращает реальный IP Cloudflare **104.16.133.229**

Локальный Unbound (127.0.0.1:2053)

Возвращает тестовый IP **192.0.2.1**

(1.3 B) Автоматический сбор зоны и создание local-zones.conf

01

Выбрана DNSSEC-подписанная зона

Для эксперимента используется зона cloudflare.com.

02

Собраны реальные DNS-записи

Скрипт collect_zone.py автоматически получает данные зоны.

03

Создано локальное описание зоны

На основе собранных данных формируется local-zones.conf для Unbound.

(1.3 Г) Подтверждение ответа из локальной базы данных

Сравнение с публичным DNS:
8.8.8.8 возвращает реальные IP Cloudflare
104.16.133.229, 104.16.132.229
локальный Unbound (127.0.0.1:2053)
возвращает
192.0.2.1

Проверка поддомена и отсутствующего имени

www.cloudflare.com → **192.0.2.2**
hack.cloudflare.com → **NXDOMAIN за 1 мс**

Анализ трафика (tcpdump)
видна только пара пакетов:

- **клиент → резолвер → клиент**
- внешних DNS-запросов нет

Логи Unbound

- docker compose logs unbound | grep -i cloudflare → **пустой вывод**
- записей resolving cloudflare.com нет

Спасибо за внимание

Написать авторам
@samoilov_ma
@Ch_alex777
@magelatova

GitHub